

title "AIに爆速で正確な図を書かせる"

MDD(Markdown with Diagrams)を使ってAIにいい感じの図を作らせる技術

図の生成はすごく便利

認知負荷を下げるために、テキストだけでは伝わりにくい情報を図にすると便利。

設計書にアーキテクチャ図を入れたい



エンジニア

提案書にフロー図で流れを見せたい



PM

仕様書に画面遷移図を付けたい



デザイナー

しかし、AIで図の生成は課題が多い

AIが思ったとおりに図を作ってくれない



悩み1

AIが1つの図を作るだけで数分かかる



悩み2

AIが作った図はバージョン管理が難しい



悩み3

そもそも

Q AIに作図させるのはなぜこんなにも難しいのか？

A AIが情報を扱う上でグラフィカルに関わる情報は全てノイズ。
pptxやdrawioなどの図をしまうデータというのは
AIにとっては不要な情報の塊でしかない。

Q ではどうすればAIは図を上手に扱えるようになるか？

A AIに図を書かせようという発想がそもそもの間違い！
図を書くのはAIの仕事ではない！ AIはテキストデータだけを扱うべき。
図が欲しいならDSLを図に変換するツールをAIに作らせた方が効率がいい！

ということで作りました。

 [ppdx999 / mdd](#)

Markdown with Diagrams — テキストから図を生成する軽量プリプロセッサ

 Rust  MIT

<https://github.com/ppdx999/mdd>

MDD とは？

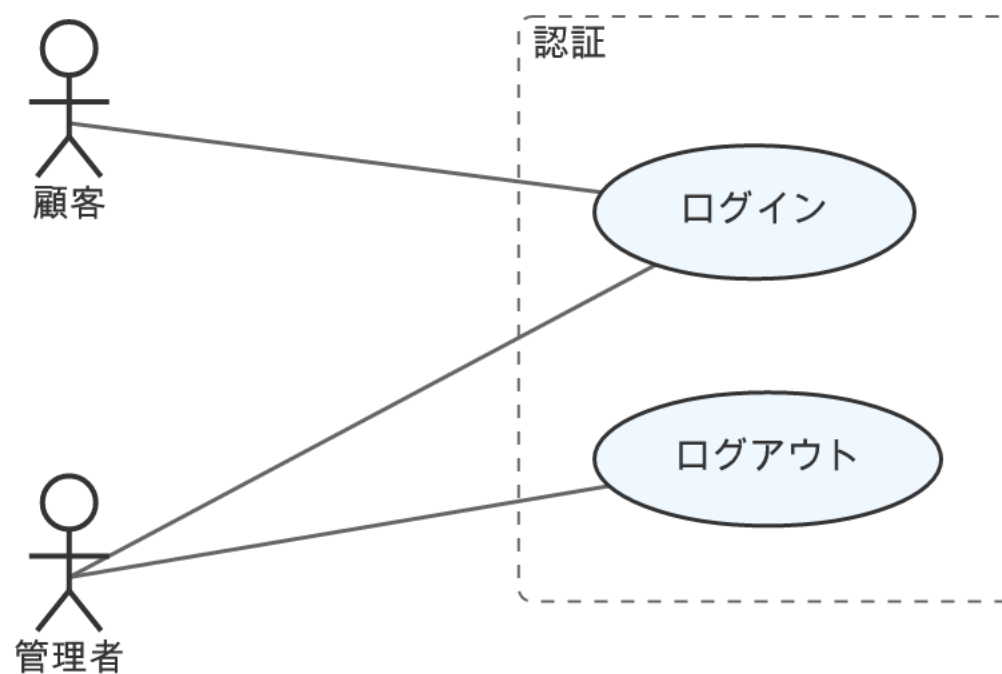
テキストからいい感じのSVGを生成するプログラム

例えば、以下のようなテキストをMDDに与えると...

```
1 actor 顧客
2 actor 管理者
3
4 package "認証" {
5   usecase ログイン
6   usecase ログアウト
7 }
8
9 顧客 -> ログイン
10 管理者 -> ログイン
11 管理者 -> ログアウト
```



これをmddが変換すると...



いい感じのユースケース図(SVG)が出力される!

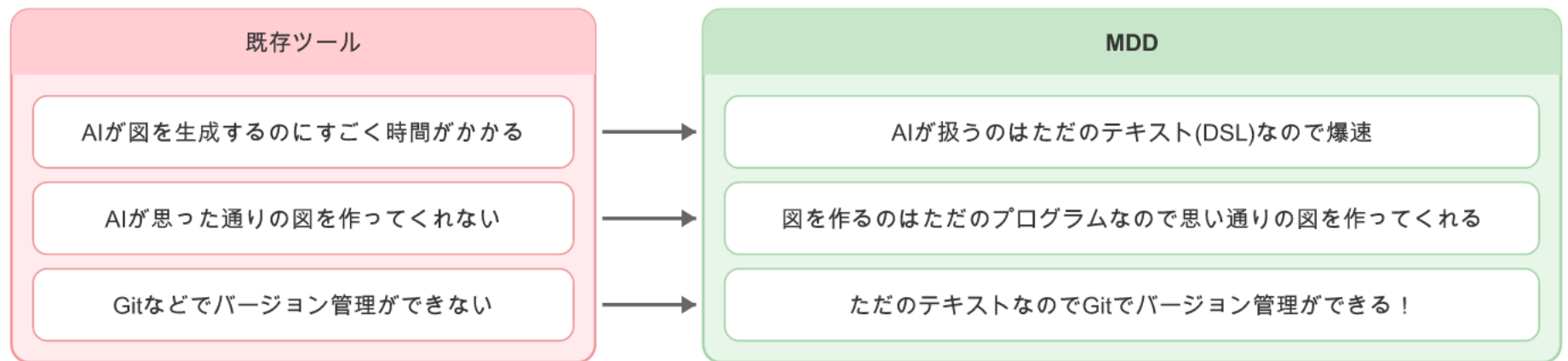
MDD の仕組み



余談

このテキストから図を作る技術は1981年に出させた論文の技術が元になっています。詳しく知りたい人はsugiyama法で検索！

この仕組みがなぜ良いのか？



どんな図が作れる？ — プロジェクト計画

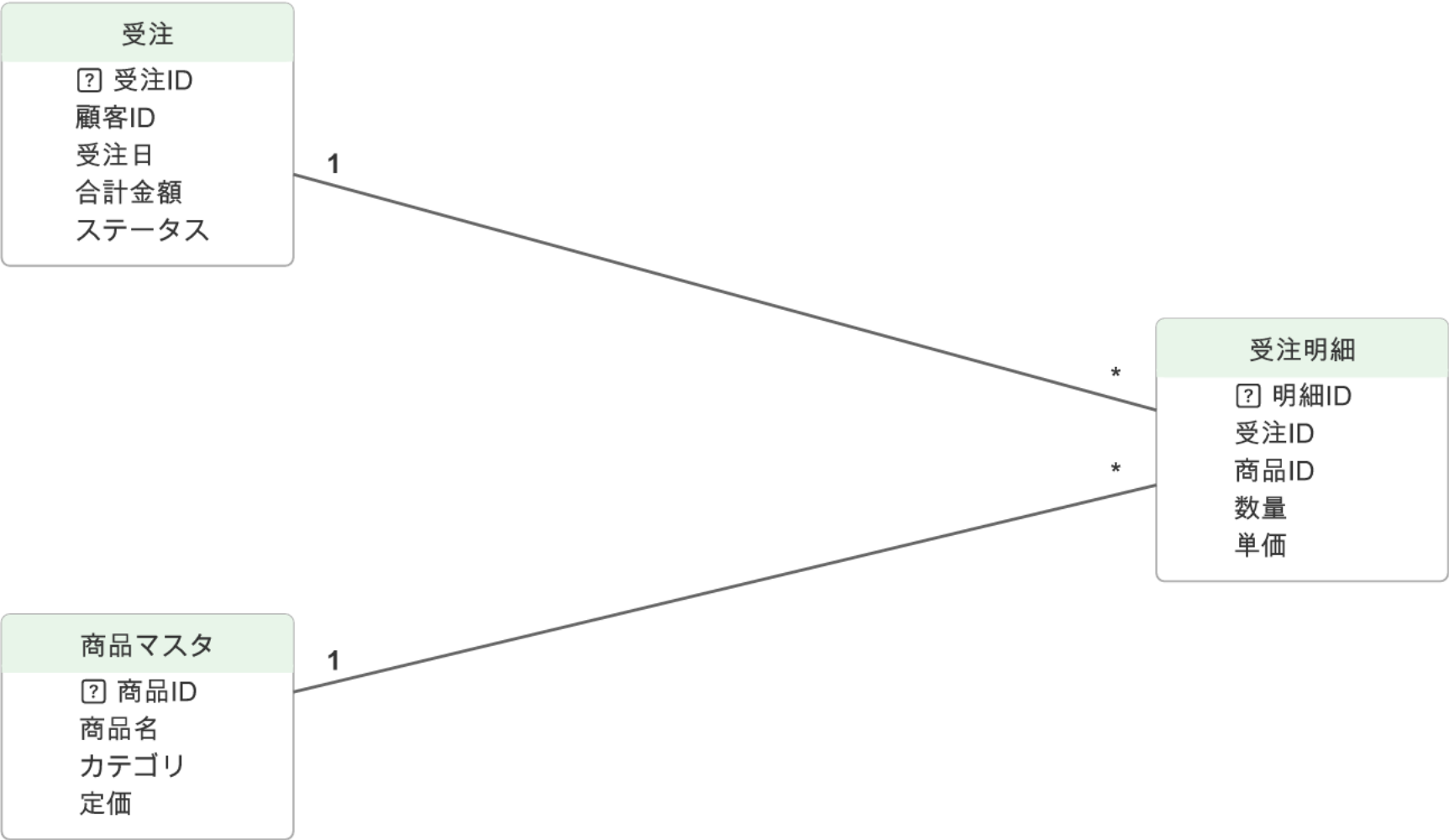
```
1 title 基幹システム刷新PJ
2 unit day
3
4 要件定義 : 2025-04-01, 3d
5 基本設計 : after 要件定義, 3d
6 詳細設計 : after 基本設計, 2d
7 開発 : after 詳細設計, 5d
8 テスト : after 開発, 3d
9 移行 : after テスト, 2d
```



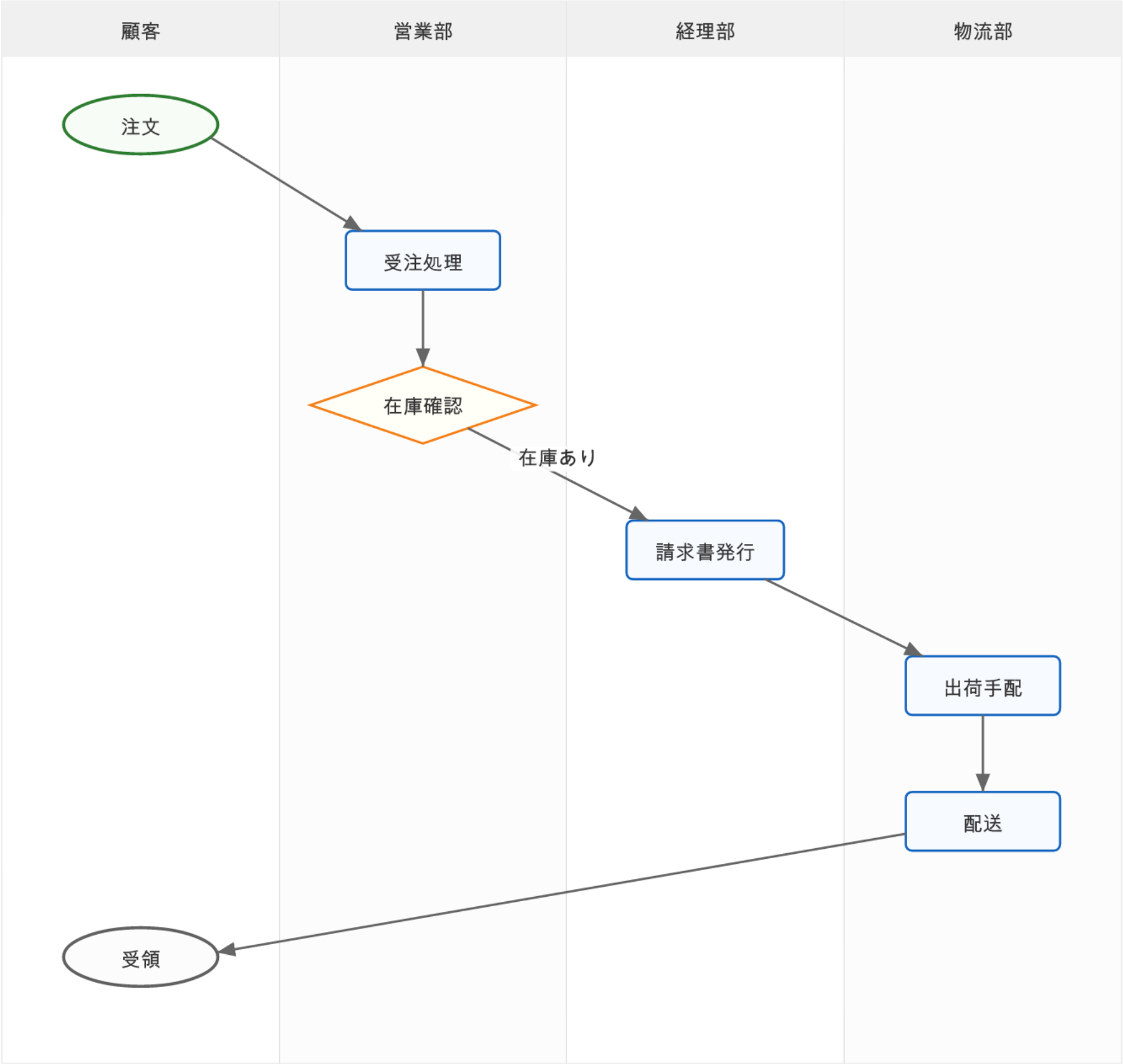
```
``gantt
title 基幹システム刷新PJ
unit day
```

```
要件定義 : 2025-04-01, 3d
基本設計 : after 要件定義, 3d
詳細設計 : after 基本設計, 2d
開発 : after 詳細設計, 5d
テスト : after 開発, 3d
移行 : after テスト, 2d
...
```

```
1  table 受注 {
2    * 受注ID
3    顧客ID
4    受注日
5    合計金額
6    ステータス
7  }
8
9  table 受注明細 {
10   * 明細ID
11   受注ID
12   商品ID
13   数量
14   単価
15 }
16
17 table 商品マスタ {
18   * 商品ID
19   商品名
20   カテゴリ
21   定価
22 }
23
24 受注 1--* 受注明細
25 商品マスタ 1--* 受注明細
```



```
1 lane 顧客
2 lane 営業部
3 lane 経理部
4 lane 物流部
5
6 顧客: start 注文
7 営業部: process 受注処理
8 営業部: decision 在庫確認
9 経理部: process 請求書発行
10 物流部: process 出荷手配
11 物流部: process 配送
12 顧客: end 受領
13
14 注文 -> 受注処理
15 受注処理 -> 在庫確認
16 在庫確認 -> 請求書発行 : "在庫あり"
17 請求書発行 -> 出荷手配
18 出荷手配 -> 配送
19 配送 -> 受領
```



どんな図が作れる？ — 組織図

```
1 title "プロジェクト体制図"
2 member PM : "プロジェクトマネージャー"
3 member AP_Lead : "APリーダー"
4 member Infra_Lead : "インフラリーダー"
5 member QA_Lead : "QAリーダー"
6 member AP1 : "AP開発メンバー"
7 member AP2 : "AP開発メンバー"
8 member Infra1 : "インフラメンバー"
9 member QA1 : "QAメンバー"
10 PM -> AP_Lead
11 PM -> Infra_Lead
12 PM -> QA_Lead
13 AP_Lead -> AP1
14 AP_Lead -> AP2
15 Infra_Lead -> Infra1
16 QA_Lead -> QA1
```



```
```org
title "プロジェクト体制図"
member PM : "プロジェクトマネージャー"
member AP_Lead : "APリーダー"
member Infra_Lead : "インフラリーダー"
member QA_Lead : "QAリーダー"
member AP1 : "AP開発メンバー"
member AP2 : "AP開発メンバー"
member Infra1 : "インフラメンバー"
member QA1 : "QAメンバー"
PM -> AP_Lead
PM -> Infra_Lead
PM -> QA_Lead
AP_Lead -> AP1
AP_Lead -> AP2
Infra_Lead -> Infra1
QA_Lead -> QA1
```
```

どんな図が作れる？ — 技術比較

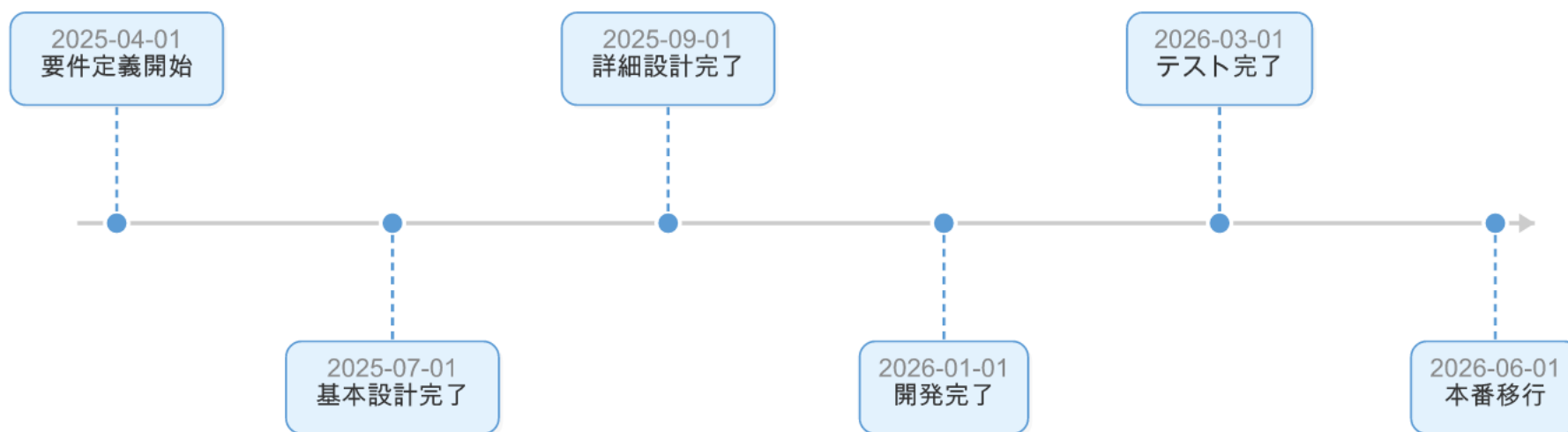
```
1  title "フレームワーク比較"
2  option "Spring Boot" {
3    言語: Java
4    実績: 豊富
5    学習コスト: 中
6    エコシステム: 大規模
7    保守性: 高い
8  }
9  option "Ruby on Rails" {
10   言語: Ruby
11   実績: 豊富
12   学習コスト: 低
13   エコシステム: 中規模
14   保守性: 中
15 }
16 option "Next.js" {
17   言語: TypeScript
18   実績: 増加中
19   学習コスト: 中
20   エコシステム: 大規模
21   保守性: 高い
22 }
```



```
```compare
title "フレームワーク比較"
option "Spring Boot" {
 言語: Java
 実績: 豊富
 学習コスト: 中
 エコシステム: 大規模
 保守性: 高い
}
option "Ruby on Rails" {
 言語: Ruby
 実績: 豊富
 学習コスト: 低
 エコシステム: 中規模
 保守性: 中
}
option "Next.js" {
 言語: TypeScript
 実績: 増加中
 学習コスト: 中
 エコシステム: 大規模
 保守性: 高い
}
...
```
```

どんな図が作れる？ — タイムライン

- 1 2025-04-01 : 要件定義開始
- 2 2025-07-01 : 基本設計完了
- 3 2025-09-01 : 詳細設計完了
- 4 2026-01-01 : 開発完了
- 5 2026-03-01 : テスト完了
- 6 2026-06-01 : 本番移行



どんな図が作れる？ — リスク分析

```
1 title "プロジェクトリスクマトリクス"
2 x-axis "影響度：小" "影響度：大"
3 y-axis "発生確率：低" "発生確率：高"
4 quadrant 1："監視"
5 quadrant 2："最優先対応"
6 quadrant 3："許容"
7 quadrant 4："軽減策検討"
```



mddが変換

```
``matrix
```

```
title "プロジェクトリスクマトリクス"
```

```
x-axis "影響度：小" "影響度：大"
```

```
y-axis "発生確率：低" "発生確率：高"
```

```
quadrant 1："監視"
```

```
quadrant 2："最優先対応"
```

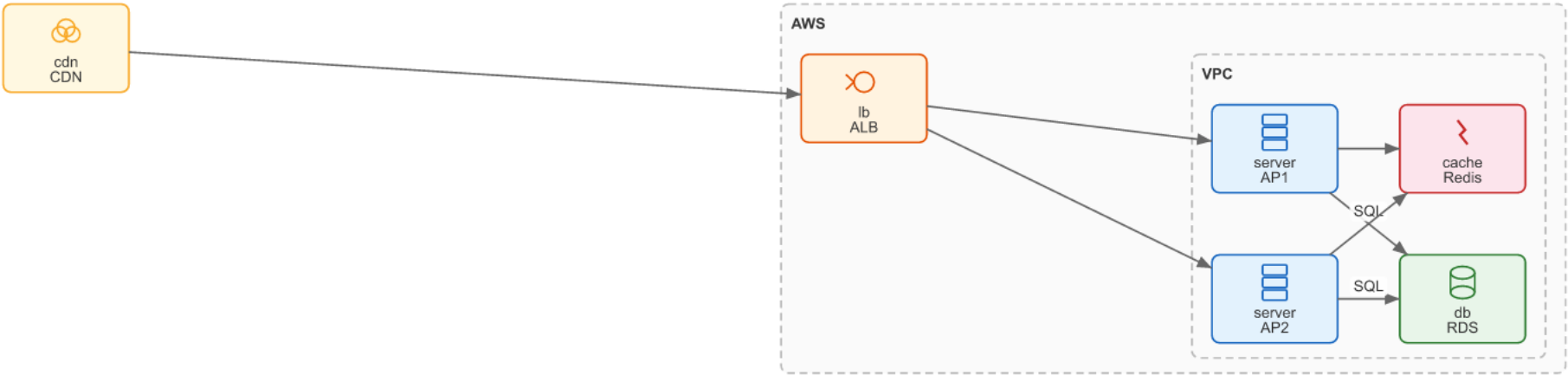
```
quadrant 3："許容"
```

```
quadrant 4："軽減策検討"
```

```
``
```

```
1  node CDN type=cdn
2
3  group "AWS" {
4    node ALB type=lb
5    group "VPC" {
6      node AP1 type=server
7      node AP2 type=server
8      node RDS type=db
9      node Redis type=cache
10   }
11 }
12
13 CDN -> ALB
14 ALB -> AP1
15 ALB -> AP2
16 AP1 -> RDS : "SQL"
17 AP2 -> RDS : "SQL"
18 AP1 -> Redis
19 AP2 -> Redis
```

↓
mddが変換



どんな図が作れる？ — SWOT分析

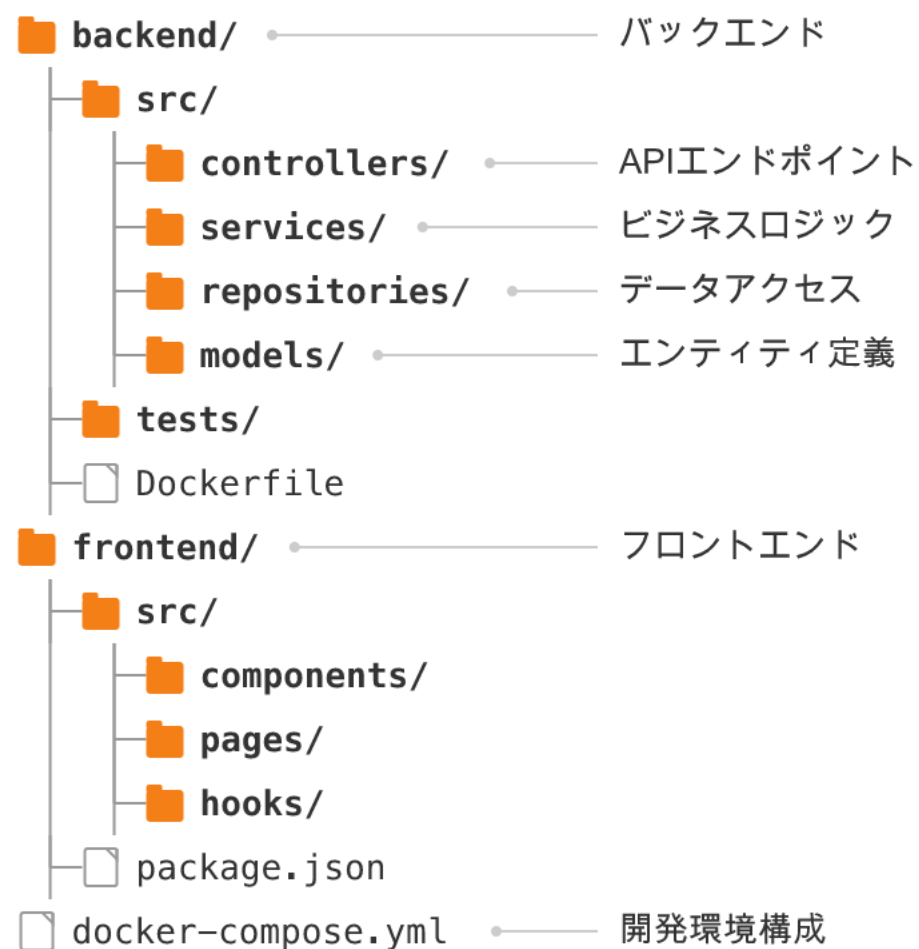
```
1  title "プロジェクト SWOT"
2  strengths {
3    チームの技術力が高い
4    既存業務知識が豊富
5  }
6  weaknesses {
7    レガシーシステムとの依存
8    テスト環境が不十分
9  }
10 opportunities {
11   クラウド移行でコスト削減
12   DXによる業務効率化
13 }
14 threats {
15   納期遅延リスク
16   要件変更の頻発
17 }
```



mddが変換

```
```swot
title "プロジェクト SWOT"
strengths {
 チームの技術力が高い
 既存業務知識が豊富
}
weaknesses {
 レガシーシステムとの依存
 テスト環境が不十分
}
opportunities {
 クラウド移行でコスト削減
 DXによる業務効率化
}
threats {
 納期遅延リスク
 要件変更の頻発
}
```
```

```
1  backend/ : "バックエンド"
2  src/
3  controllers/ : "APIエンドポイント"
4  services/ : "ビジネスロジック"
5  repositories/ : "データアクセス"
6  models/ : "エンティティ定義"
7  tests/
8  Dockerfile
9  frontend/ : "フロントエンド"
10 src/
11 components/
12 pages/
13 hooks/
14 package.json
15 docker-compose.yml : "開発環境構成"
```



(おまけ) このスライドの正体

このスライド自体が、ただの Markdown テキストから自動生成されています。

```
1  title "INTRODUCTION.md"
2  ---
3  # MDD — Markdown with Diagrams
4
5  ドキュメントに図を入れたい。でも、もっと簡単に。
6
7  # 図の生成はすごく便利
8
9  ```usecase
10 actor 顧客
11 actor 管理者
12 package "認証" {
13   usecase ログイン
14 }
15 顧客 -> ログイン
```